EDD MedTrack

Fase 3 — Trazabilidad, Rutas y Seguridad

Manual Técnico

Estructuras de Datos — USAC

Arquitectura extendida: grafo no dirigido de colaboración, tabla hash por tipo de usuario, compresión LZW para mensajería, y nuevos endpoints REST para chat y colaboración.

Autor: Angel Raúl Herrera Chilel
Carnet: 202402587
Sección: B

Proyecto: EDD_Proyecto1_Fase3
Fecha: 6 de mayo de 2026

Índice

Introducción **2**

1 Nuevas Estructuras de Datos **2**
1.1 Grafo No Dirigido – `gestión_datos/grafo.pm` 2
1.2 Tabla Hash – `gestión_datos/tabla_hash.pm` 2
1.3 Compresión LZW – `utilidades/lzw.pm` 3
1.4 Persistencia de Chats – `persistencia/chats.pm` 3

2 Nuevos Módulos de la API **3**
2.1 `api::colaboracion.pm` 3
2.2 `api::chat.pm` 3
2.3 `api::reportes.pm` 4
2.4 `api::admin.pm` y `api::user.pm` (extensiones) 4

3 Carga y Persistencia de Relaciones **4**

4 Middleware y Sesiones **4**

5 Generación de Reportes Graphviz (Nuevos) **4**

6 Integración con Frontend React **5**

7 Consideraciones de Seguridad **5**

Introducción

Este manual describe la implementación de la Fase 3 del sistema EDD MedTrack, donde se incorporan estructuras avanzadas: grafo no dirigido para la red de colaboración entre profesionales, tabla hash para consultas instantáneas por tipo de usuario, y compresión LZW para la persistencia eficiente de los chats. Se detalla la integración con las estructuras previas (AVL, BST, B-Tree, listas) y la nueva API REST habilitada para el frontend React.

Nuevas Estructuras de Datos

Grafo No Dirigido – gestión_datos/grafo.pm

Modela la red de colaboración activa entre profesionales. Cada nodo se identifica por el número de colegio y almacena nombre, departamento, tipo y especialidad. Las aristas son no dirigidas y solo existen entre relaciones con estado `ACTIVA`.

Representación interna:

- `$self->{nodos}` : hash {colegio =>{nombre, departamento, tipo, especialidad}}
- `$self->{adyacencia}` : hash de hash para vecinos inmediatos
- `$self->{aristas}` : hash con clave normalizada "colegio_a|colegio_b"

Operaciones:

Método	Descripción
`agregar_nodo(...)`	Registra un profesional en el grafo
`agregar_arista(a,b)`	Crea conexión (solo si ambos nodos e
`obtener_vecinos(nodo)`	Retorna lista de colegios adyacentes (
`sugerencias_bfs(origen)`	BFS de dos saltos: devuelve candidato nados por colaboradores en común
`son_vecinos(a,b)`	Verifica si ya son colaboradores direct
`actualizar_departamento(colegio, nuevo)`	Cambia atributo del nodo (útil para nación)

Tabla Hash – gestión_datos/tabla_hash.pm

Organiza al personal agrupándolo por su tipo de usuario (`TIPO-01` a `TIPO-05`). Cada bucket es un array que almacena referencias a los nodos del AVL.

Características: - Tamaño fijo de 5 buckets (uno por tipo). - Función hash directa: `_indice_tipo(tipo)` - Inserción/eliminación en tiempo constante O(1) promedio. - Registra colisiones cuando un bucket tiene más de un elemento.

Métodos públicos:

- `insertar($usuario)` – agrega o actualiza
- `buscar_por_tipo($tipo)` – retorna lista de usuarios de ese tipo
- `contar_por_tipo($tipo)` – número de profesionales
- `obtener_estadisticas()` – devuelve ocupación y colisiones por bucket
- `reconstruir_desde_avl($avl)` – barre el AVL y llena la tabla

Compresión LZW – utilidades/lzw.pm

Implementa los algoritmos de compresión y descompresión para los historiales de chat.
Funciones:

```
sub comprimir($texto)              # devuelve arrayref de códigos enteros
sub descomprimir($codigos_ref)     # devuelve string reconstruido
sub comprimir_archivo($origen, $destino)    # escribe binario .lzw
sub descomprimir_archivo($origen, $destino) # lee .lzw y genera texto
```

El formato de persistencia: archivo `colegio.lzw` con cabecera binaria generada por `pack('N*', @codigos)`. La descompresión convierte los enteros de 32 bits de vuelta a texto usando el mismo diccionario inicial (0-255 = caracteres ASCII).

Persistencia de Chats – persistencia/chats.pm

Cada usuario tiene su propio archivo comprimido. La estructura en memoria es un hash: `$chats{colegio_destino} = [ mensaje1, mensaje2, ... ]`. Cada mensaje incluye `remitente`, `receptor`, `contenido`, `timestamp`.

Flujo de trabajo:

1. `cargar_chats($colegio)` – descomprime y decodifica JSON.
2. `agregar_mensaje($colegio, $conversacion, $mensaje)` – añade en memoria y guarda inmediatamente (cada mensaje persiste).
3. `guardar_chats($colegio, $conversaciones)` – serializa, comprime con LZW y escribe.

Nuevos Módulos de la API

api::colaboracion.pm

Expone endpoints para gestionar la red de colaboración:

- `colaboradores_actuales()` – vecinos directos en el grafo.
- `sugerencias()` – BFS de dos saltos ordenado por afinidad.
- `enviar_solicitud_colaboracion()` – crea relación con estado `PENDIENTE` en `relaciones.json`.
- `solicitudes_recibidas()` – lista de remitentes con solicitud pendiente.
- `responder_solicitud()` – acepta (crea arista en grafo) o rechaza.

api::chat.pm

Maneja la mensajería:

- `enviar_mensaje()` – valida que sean vecinos en el grafo, luego guarda en ambos archivos (emisor y receptor).
- `obtener_mensajes()` – recupera conversación con un colaborador, soporta polling incremental mediante timestamp `desde`.
- `conversaciones_recientes()` – lista de colegios con los que se ha chateado, ordenada por último mensaje.

api::reportes.pm

Genera reportes avanzados:

- `grafo_colaboraciones()` – produce código DOT, ejecuta `dot -Tpng` y retorna ruta de la imagen.
- `lista_adyacencia()` – representación textual de adyacencia del grafo.
- `tabla_hash_estado()` – estadísticas de la tabla hash.
- `reporte_compresion()` – lista los archivos `.lzw` en `datos/chats/`.

api::admin.pm y api::user.pm (extensiones)

- `asignar_departamento()` – actualiza el AVL, la tabla hash y el atributo del nodo en el grafo.
- `solicitar_insumo()` – registra petición en lista circular doblemente enlazada (`$solicitudes`).
- `inventario_suministros()` – consulta en el árbol B (orden 4) con filtro opcional.

Carga y Persistencia de Relaciones

El archivo `datos/relaciones.json` almacena todas las solicitudes de colaboración, con estructura:

```
[
  {
    "colegio_a": "COL-10001",
    "colegio_b": "COL-20001",
    "estado": "ACTIVA"
  }
]
```

En el arranque, `persistencia::relaciones::inicializar_grafo_desde_relaciones()` recorre este archivo y agrega aristas solo para aquellas con estado `ACTIVA`.

Las solicitudes pendientes (`estado = "PENDIENTE"`) no crean aristas hasta que el usuario receptor las acepta desde la interfaz.

Middleware y Sesiones

Se utiliza `Mojolicious::Lite` con sesiones firmadas (secreto `eddtrack_secret_2026_para_sesiones`). El helper `ctx` inyecta todas las estructuras de datos y el usuario autenticado en cada controlador.

El hook `before_dispatch` configura CORS para aceptar peticiones desde `http://localhost:4321` (frontend React).

Generación de Reportes Graphviz (Nuevos)

```perl
# reportes/grafo_colaboracion_dots.pm
sub generar_reporte {
    my ($grafo, $ruta_dot, $ruta_png, $ruta_svg) = @_;
    # Escribe DOT con nodos coloreados por departamento
    # Ejecuta "dot -Tpng $dot -o $png"
}
```

Los colores de los nodos en el grafo:

- `DEP-ADM` → rojo (#fca5a5)
- `DEP-MED` → verde (#86efac)
- `DEP-CIR` → azul (#7aa2f7)
- `DEP-LAB` → amarillo (#fde047)
- `DEP-FAR` → morado (#c084fc)
- `SIN-DEP` → gris (#9ca3af)

Integración con Frontend React

El backend expone una API REST en el puerto 3000. El frontend (Vite + React + TypeScript) se comunica a través de `axios` con `withCredentials: true` para mantener la sesión. Los endpoints de reportes devuelven rutas relativas a los archivos PNG generados en `reportes/`.

Para el chat, el frontend realiza polling cada 3 segundos al endpoint `/api/chat/mensajes` con el parámetro `desde` igual al timestamp del último mensaje recibido.

Consideraciones de Seguridad

- Las contraseñas se almacenan en texto plano en el JSON de usuarios (por simplicidad del proyecto).
- Las sesiones son gestionadas por Mojolicious con cookie firmada.
- La compresión LZW no es un mecanismo de cifrado, solo reduce tamaño.
- Los mensajes de chat se guardan en disco sin encriptar; en un entorno real se recomendaría cifrado.